Data Structures and Algorithms (CS1.201)

Makeup Examination (13 Aug, 2025)
Total Marks: 50 (10, 10, 10, 9+2+9)

Puzzle 1. Consider a binary search tree (BST) where two nodes of the BST are swapped by mistake. You need to correct this to get a correct BST. You can only swap the wrong values, but not change the structure of the BST. Provide an algorithm (along with its complexity) to detect and swap the wrong nodes, and produce a correct BST.

Input: A pointer to the root of a BST whose two nodes are swapped.
Output: Fix the tree in-place (modify the node values, not the structure) to restore the BST property.

Puzzle 2. You are given a directed graph with n nodes (numbered from 1 to n) and m edges. Determine whether it is possible to travel from every node to every other node (that is, whether the graph is strongly connected). Provide the algorithm, its logic, and complexity.

Puzzle 3. You are given an array of integers. For each element in the array, find the next greater element; that is, the first element to its right that is greater than the current element. If there is no greater element, then put -1. Here is an example.

Sample Input: $9, 4, 5, 7, 10, 8$
Sample Output: $10, 5, 7, 10, -1, -1$

Puzzle 4. Your friend claims he/she has correctly computed the shortest path distances from a vertex s to all other vertices in a directed graph with positive edge weights. Devise a linear-time algorithm DISTANCECHECK to check if your friend is correct.

DISTANCECHECK$(G, s, \mathbf{w}, \mathbf{d})$

DISTANCECHECK$(G, s, \mathbf{w}, \mathbf{d})$ takes as input a directed graph G (with vertex set V and edge set E, provided as an adjacency list), a start vertex $s \in V$, a weight function $\mathbf{w} : E \to \mathbb{N}$ on the edges of G, and a distance function $\mathbf{d} : V \to \mathbb{N}$ on the vertices of G. That is, for every edge $e \in E$, its weight $\mathbf{w}(e)$ is a positive integer, and for every vertex $v \in V$, its distance $\mathbf{d}(v)$ is a positive integer. Your algorithm's output should be "YES" if for every vertex $v \in V$ of the graph, $\mathbf{d}(v)$ is equal to the correct shortest path distance from s to v computed according to weight function $\mathbf{w}$; and it should be "NO" if there is at least one vertex $v \in V$ in the graph for which $\mathbf{d}(v)$ is not equal to the shortest path distance from s to v computed according to weight function $\mathbf{w}$.

(a) Write down the complete code/pseudo-code of your algorithm, clearly explaining its working in a step-by-step manner. Also provide a proof of correctness of your algorithm; that is, provide a logical reason why the algorithm correctly detects whether all the shortest path distances are actually correct.

(b) Prove that the running time of your algorithm is $O(|V| + |E|)$.

(c) You run your algorithm on your friend's graph as input, and its output is "NO". So you tell your friend about it. Then your friend removes some edges from the graph until the graph no longer contains any directed cycles; that is, your friend's graph is now a **directed acyclic graph**. Your friend asks you to fix all the wrong shortest path distances in this new graph in linear time (the graph is provided in the same format as earlier). Write down the complete code/pseudo-code to rectify all the wrong shortest path distances in the graph in $O(|V|+|E|)$ time, clearly explaining its working in a step-by-step manner.